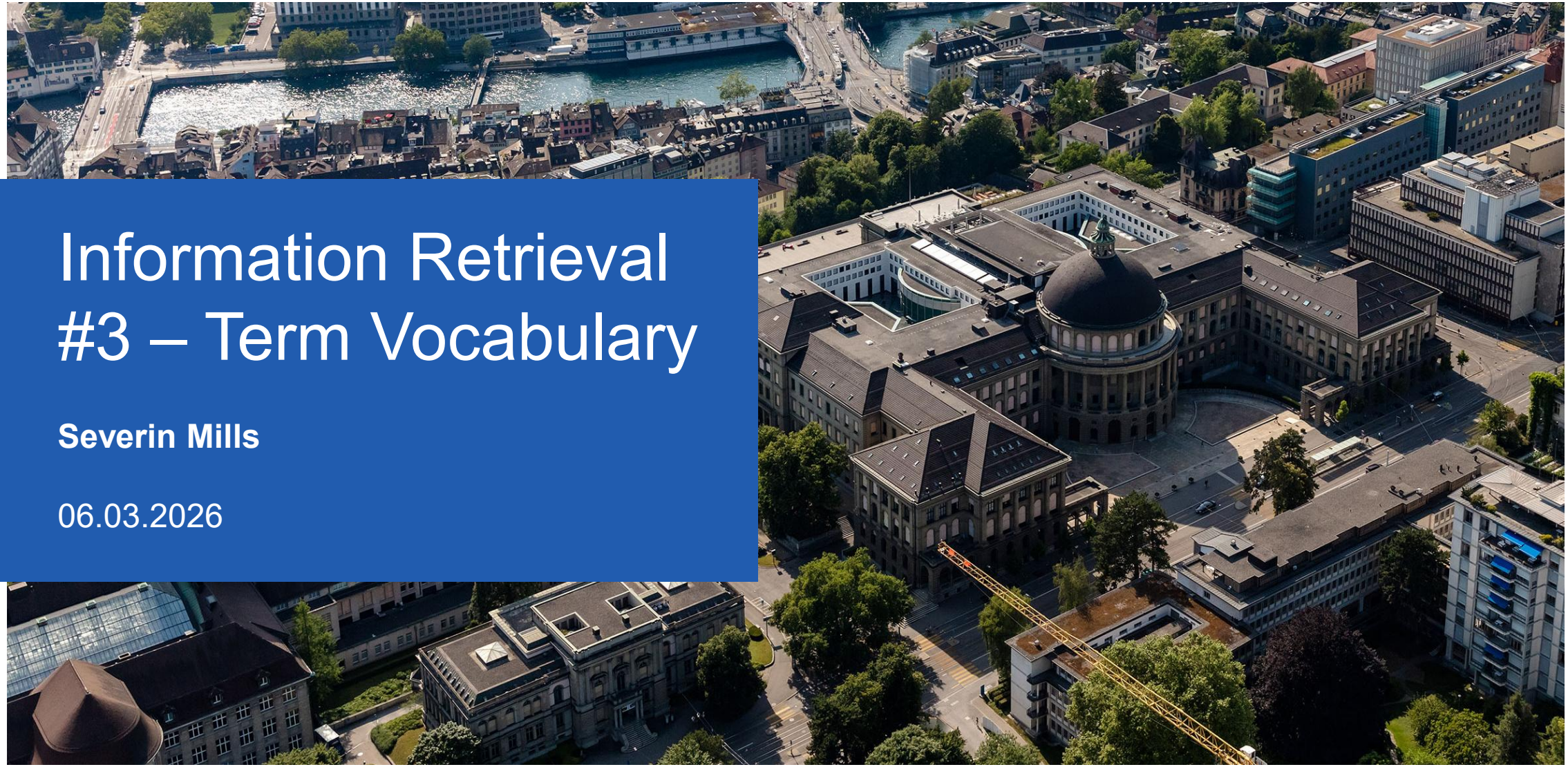
ETH zürich



Information Retrieval #3 – Term Vocabulary

Severin Mills

06.03.2026

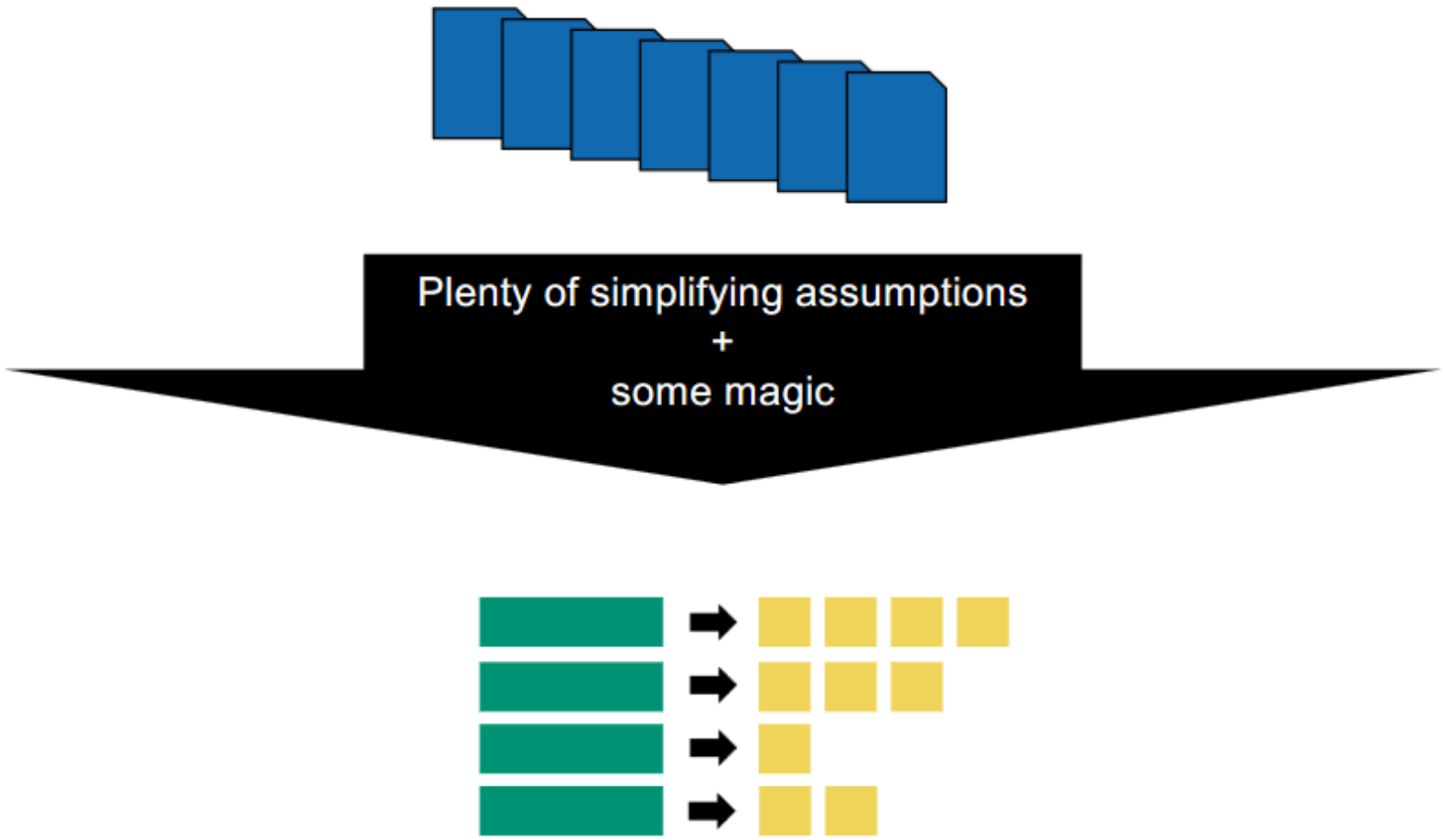
Agenda

1. Lecture Recap
2. Exercise 2: Term Vocabulary
3. Exam Questions
4. (Kahoot)

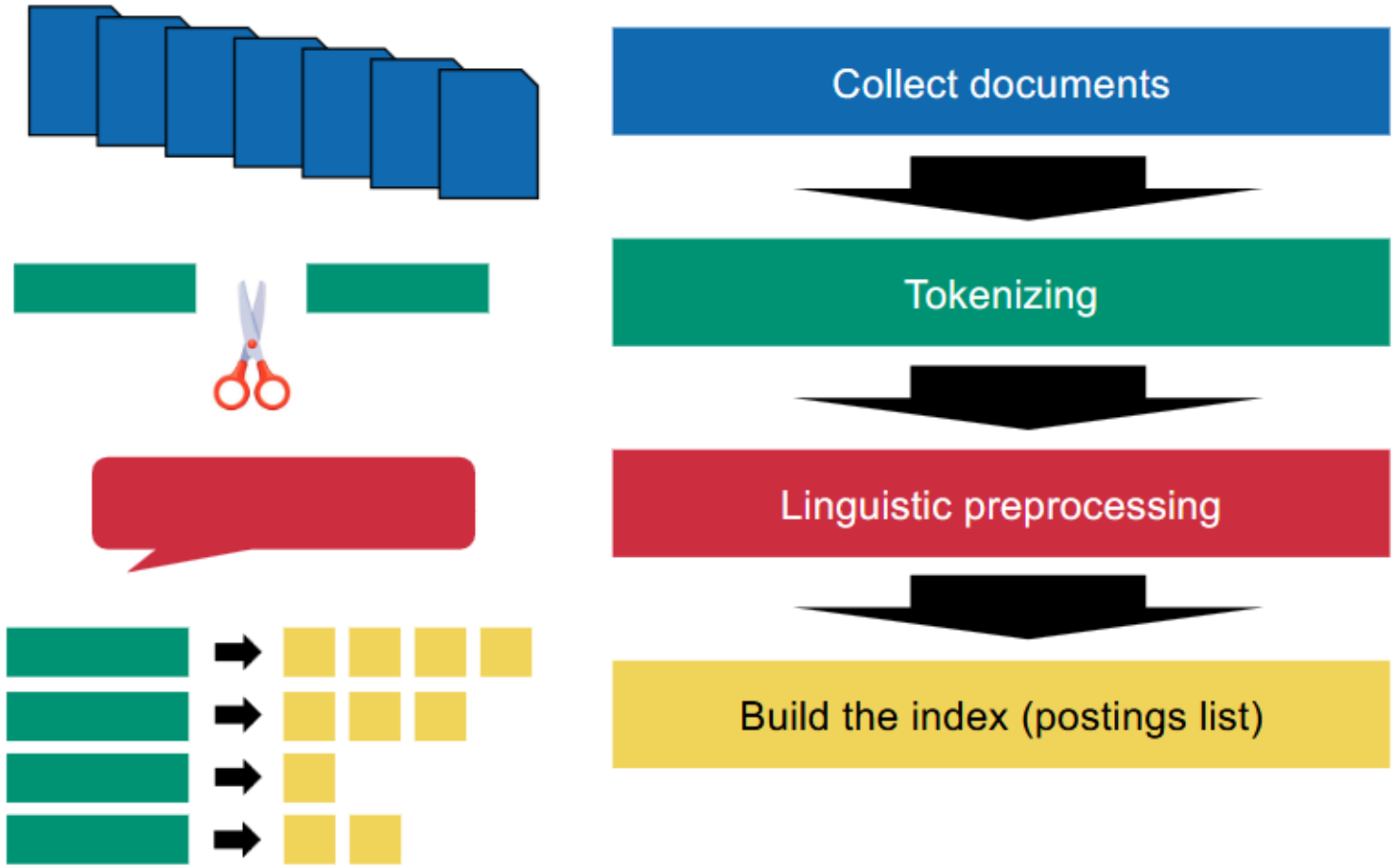
Website

Slides published here: https://www.smills.ch/

Last time



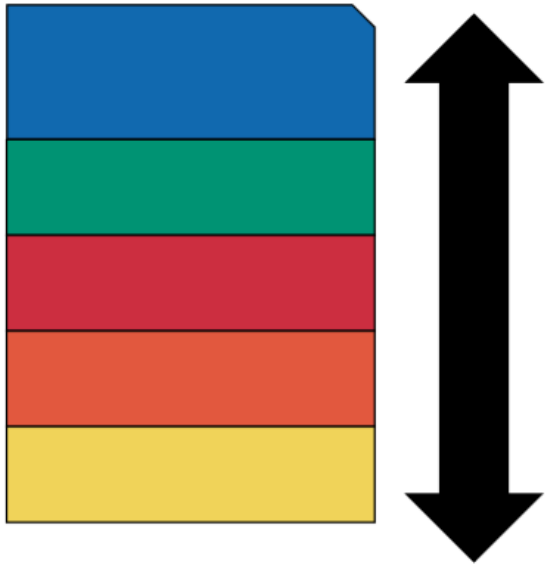
In reality…


Collect documents
Tokenizing
Linguistic preprocessing
Build the index (postings list)

Collecting Documents

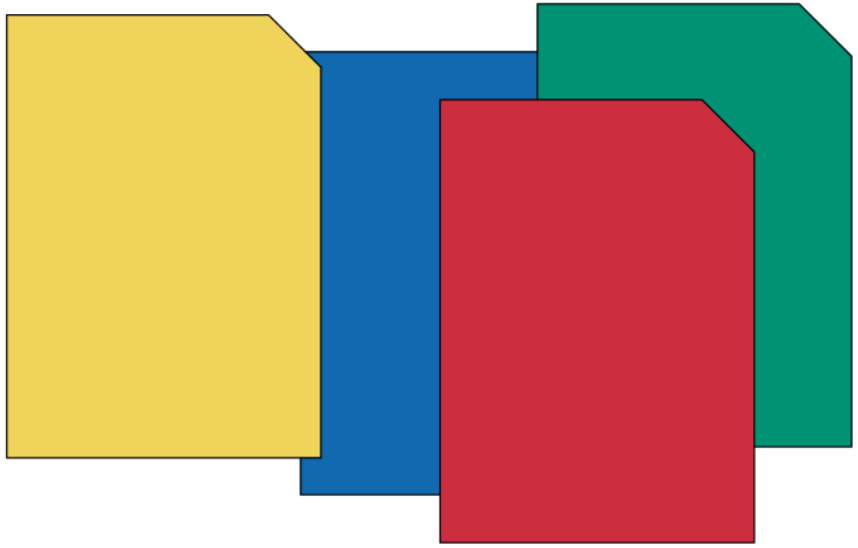
Encoding

Collecting Documents



How do we split up a document?

What about a document consisting of multiple files? (e.g. LaTeX source)



Tokenizing

- Throw away punctuation, symbols
  - Issues?
- Remove stop words

3/20/91
20/3/91
Mar 20, 1991
B-52
100.2.86.144
(800) 234-2333
800.234.2333

Hewlett-Packard
State-of-the-art
co-education
the hold-him-back-and-drag-him-away maneuver
data base
San Francisco
Los Angeles-based company
cheap San Francisco-Los Angeles fares York University vs. New York University

Tokenizing

- How to tokenize different languages

<Frustration>
Past 3rd/3rd
neghyaghtughyugumayaghpetaallu
eat go to want also
inferential
evidential
(turns out)

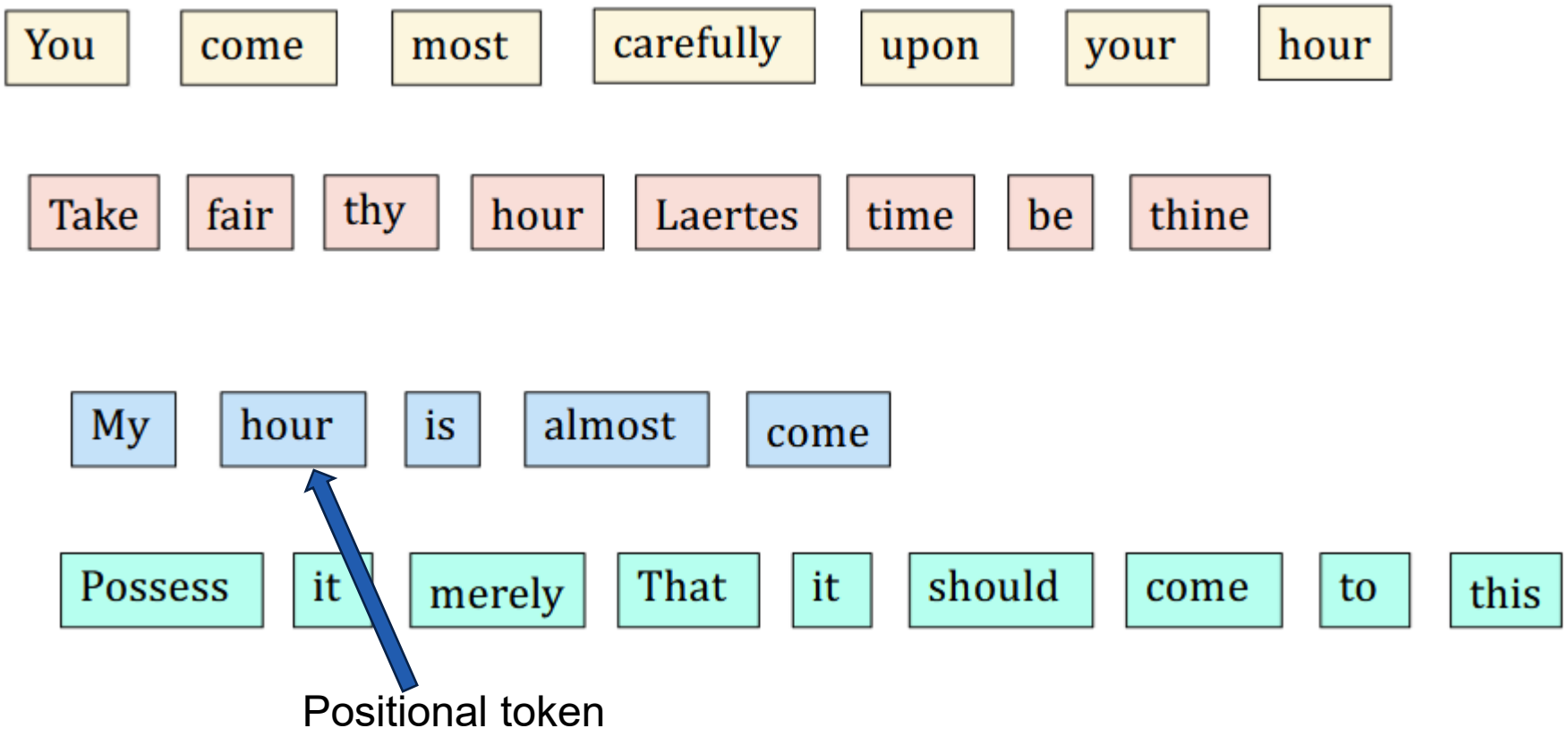
I **would** like a coffee
You **would** like a coffee
He **would** like a coffee
We **would** like a coffee
You **would** like a coffee
They **would** like a coffee

Je **voudrais** un café
Tu **voudrais** un café
Il/elle **voudrait** un café
Nous **voudrions** un café
Vous **voudrienz** un café
Ils/elles **voudraient** un café

मुझे एक कॉफी चाहिए
तुमको एक कॉफी चाहिए
उसे एक कॉफी चाहिए
हमें एक कॉफी चाहिए
आपको एक कॉफी चाहिए
उन्हें एक कॉफी चाहिए

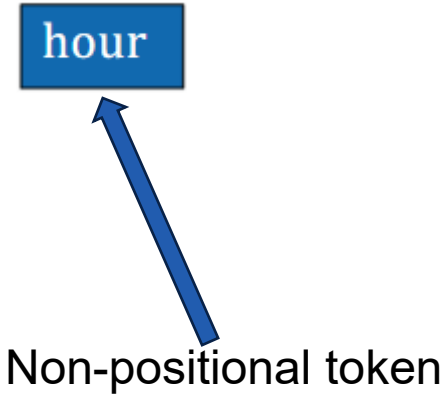
Tokenizing

- Terminology



Tokenizing

- Terminology



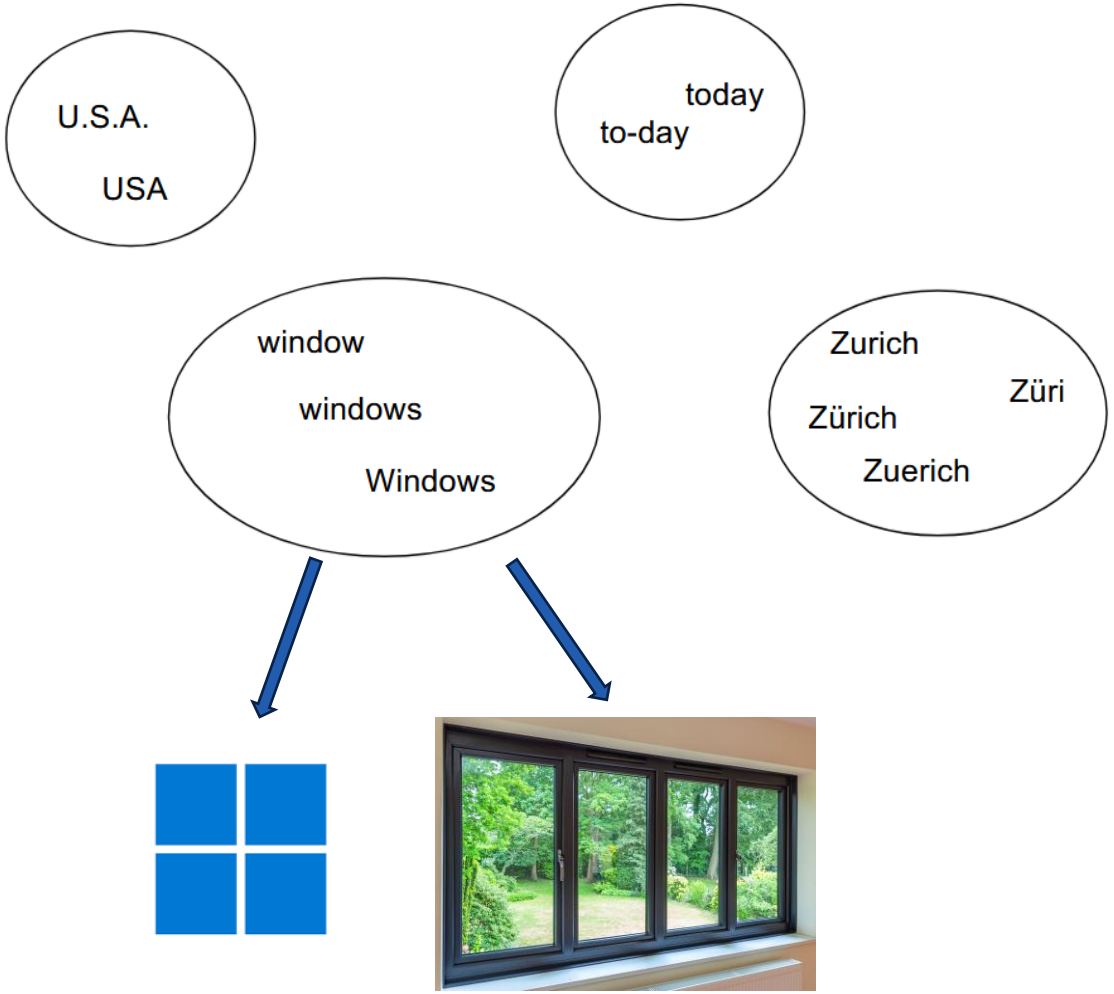
Tokenizing

- Terminology
  - Normalized types: «is» -> «be»
  - Processed: e.g. normalization

Raw or processed	Tied to document	Full name	Simplified/casual
raw	tied with position	positional token	token (implicitly positional)
raw	tied without position	non-positional token	
raw	not tied	word, non-normalized type	type (implicitly non-normalized in the book) token (compiler community)
processed	tied with position	positional posting	
processed	tied without position	non-positional posting	posting (implicitly non-positional)
processed	not tied	normalized type, term (if in index)	

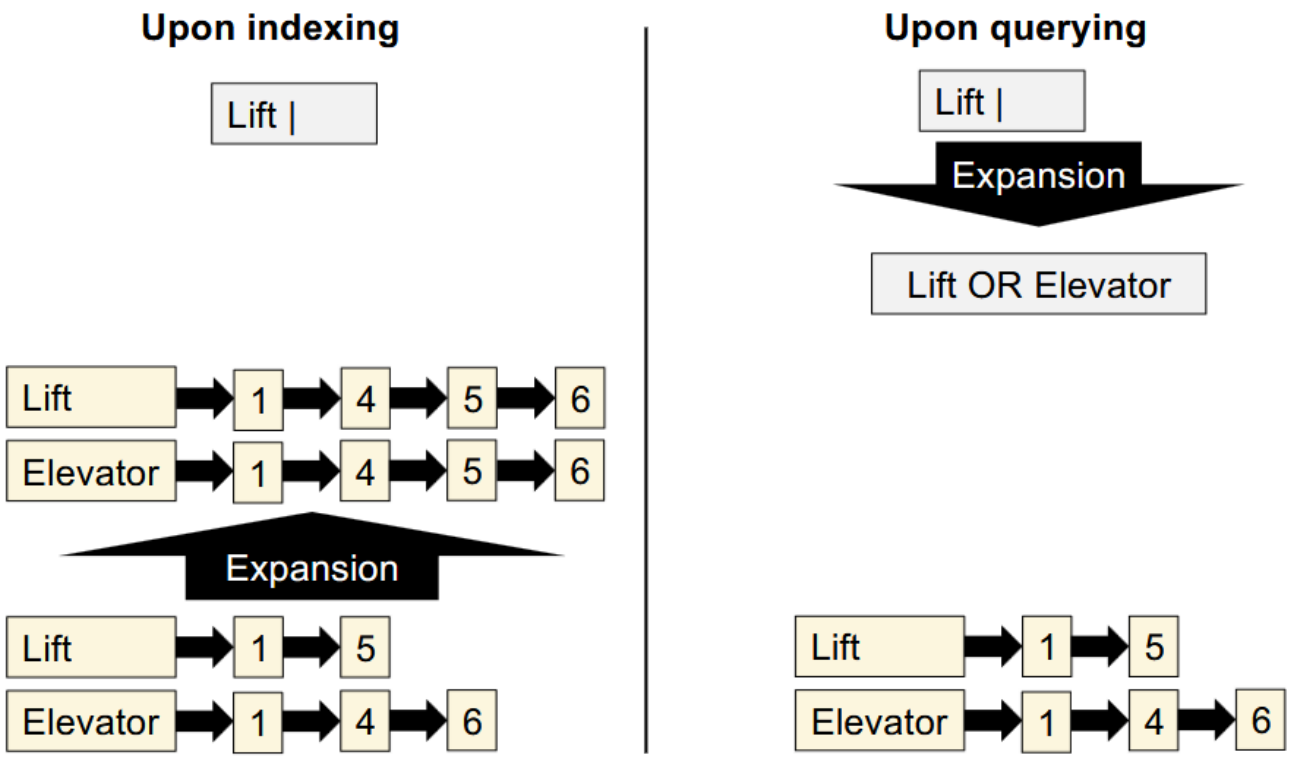
Linguistic preprocessing

- Create equivalence classes
  - Issues?
- Normalization
  - Lowercasing («Apple» -> «apple»)
  - Removing characters («U.S.A.» -> «USA»)
  - Removing accents and diacritics



Linguistic preprocessing

- Expansion
  - Union similar words (e.g. «Lift» and «Elevator»)
  - When to expand?



Stemming

- «Chop» letters of the words

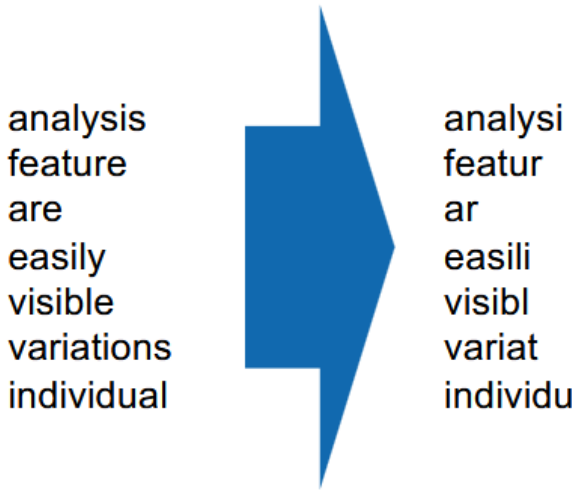
Such an analysis can reveal features that are not easily visible from the variations in the individual genes and can lead to a picture of expression that is more biologically transparent and accessible to interpretation

Porter
such an analysi can reveal featur that ar not easili visibl from the variat in the individu gene and can lead to a pictur of express that is more biolog transpar and access to interpret

Lovins
such an analys can reve featur that ar not eas vis from th vari in th individu gen and can lead to a pictur of expres that is mor biolog transpar and acces to interpres

Paice
such an analys can rev feat that are not easy vis from the vary in the individ gen and can lead to a pict of express that is mor biolog transp and access to interpret

Source: the textbook (Introduction to Information Retrieval)



"Chop" letters of the word

Lemmatization

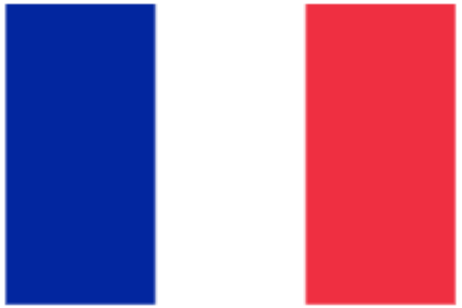
- Morphological analysis of the words
- Meaning of word considered
- E.g. better -> good

```
computer
compute
computes
computed
computation
computing
...
```

Lemmatization vs. Stemming

- When to use what?
  - Lemmatization can help with languages with more morphology

Stemming

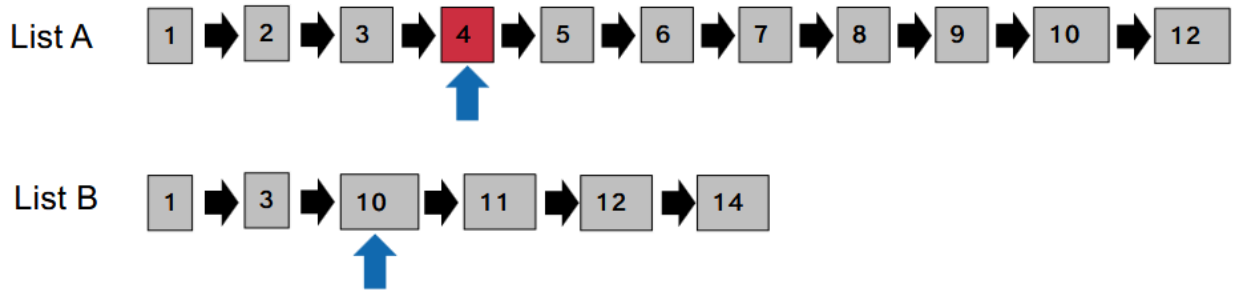


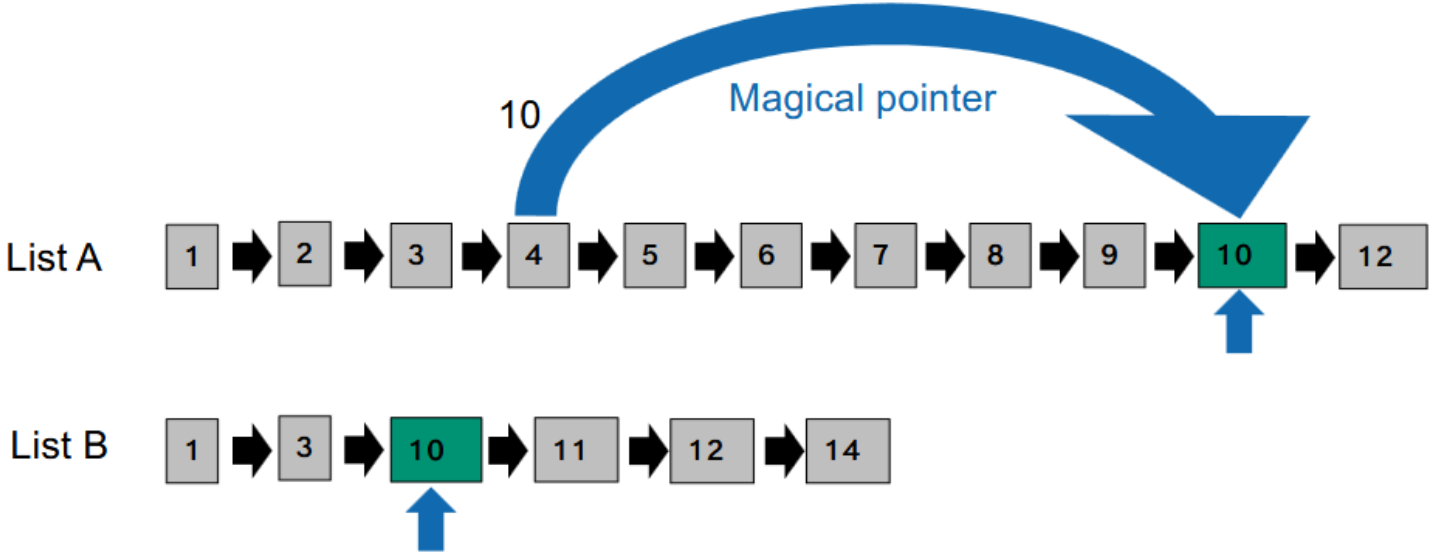
Lemmatization

Lemmatization

Skip lists

- Intersection algorithm
  - How to make this faster?

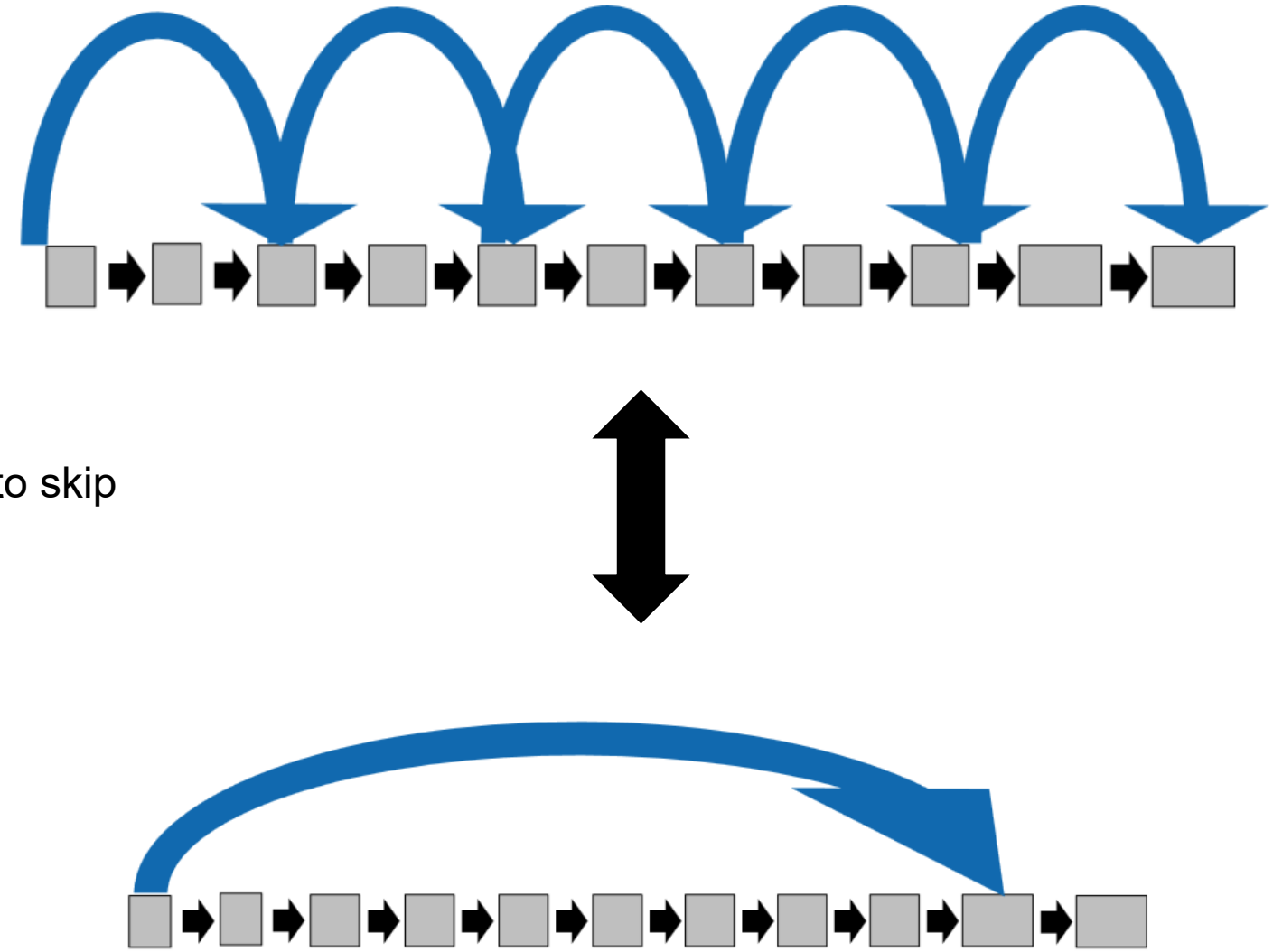




Skip lists

- Skip lists!
  - Too short skips
    - Many comparisons
    - Waste of space
  - Too long skips
    - Few comparisons
    - Not many real opportunities to skip

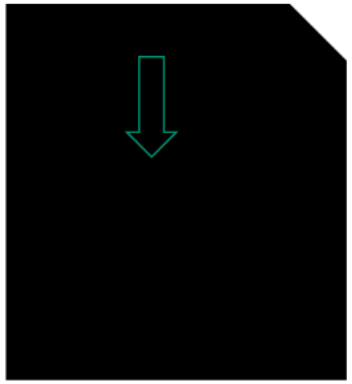
- Best: $\sqrt{\#\text{postings}}$



Phrase search

- Real world: one-word queries are uncommon
  - e.g. «Information Retrieval» vs. «Information» AND «Retrieval»
- Why do inverted indices fail?
  - No information on proximity!

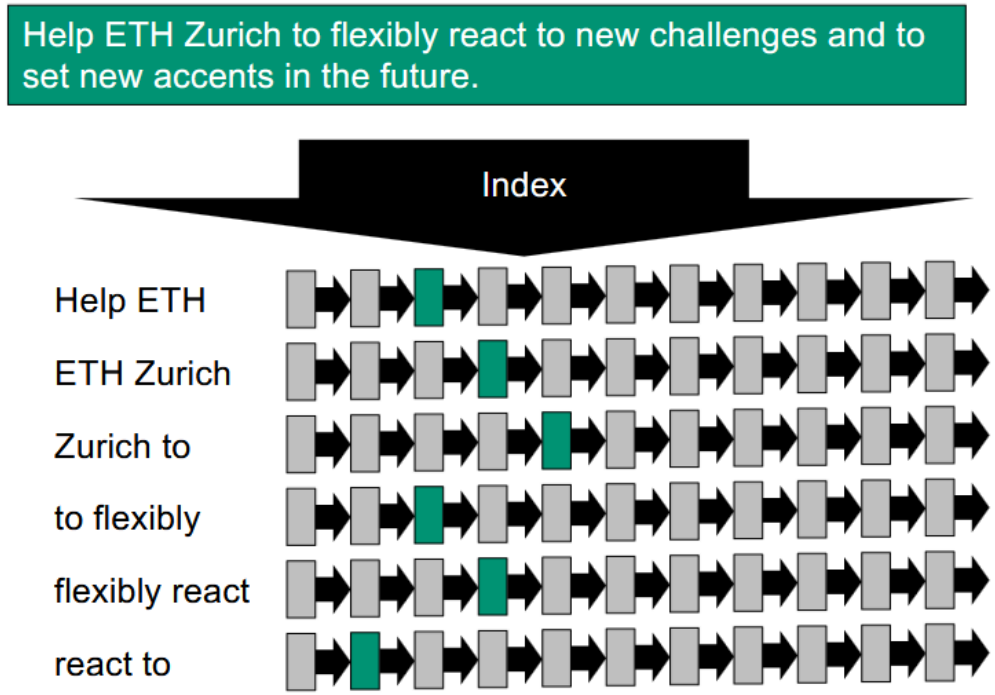
Biword indices

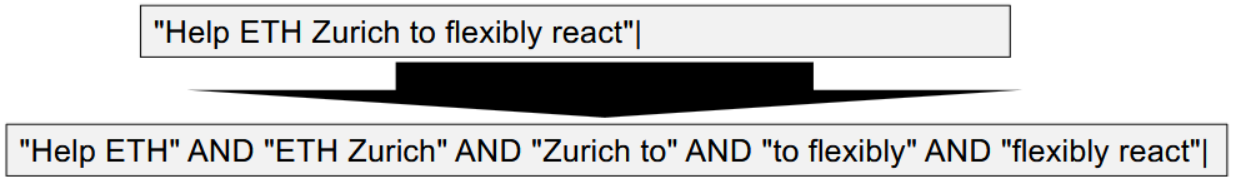


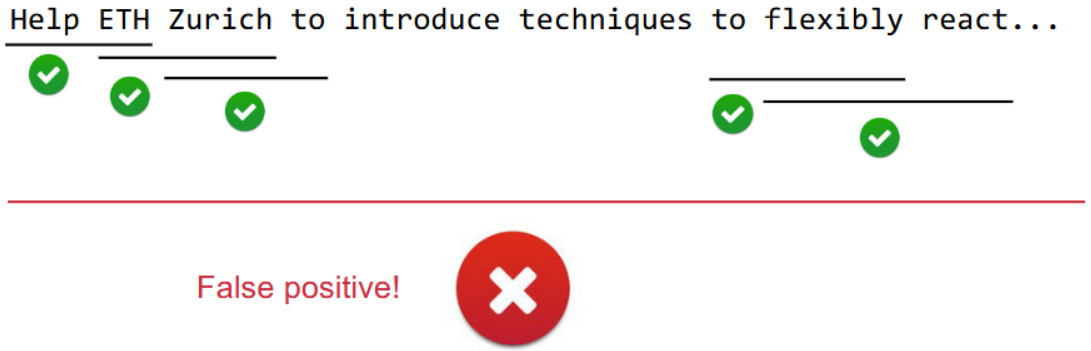
Positional indices

Bi-word indices

- Keep all pairs of words

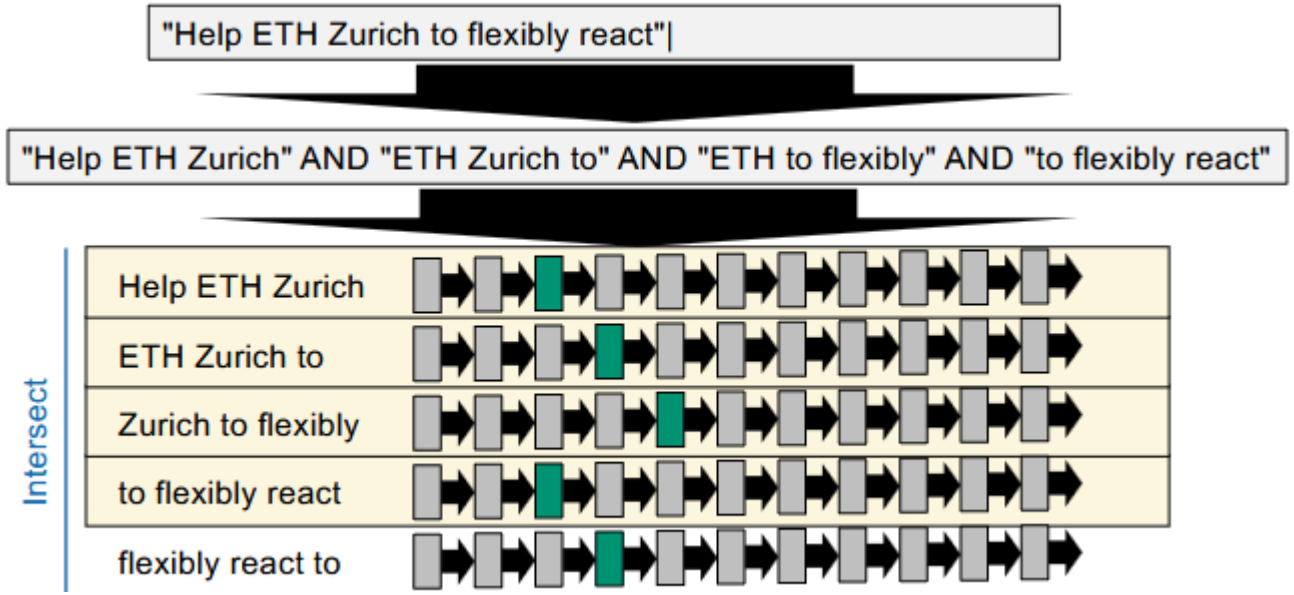


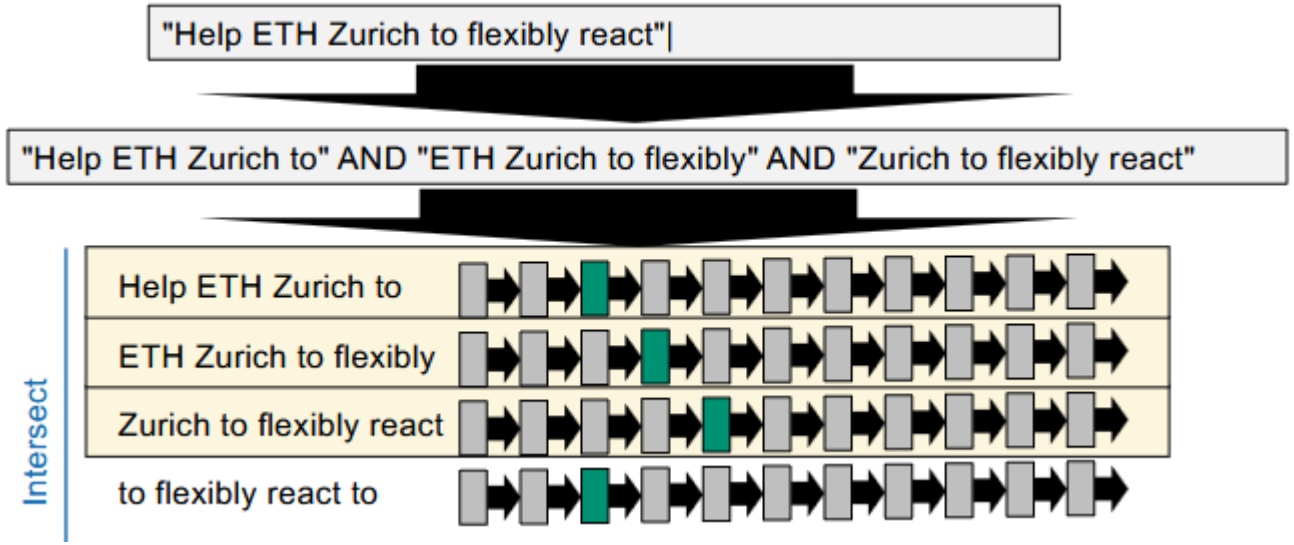




K-word indices

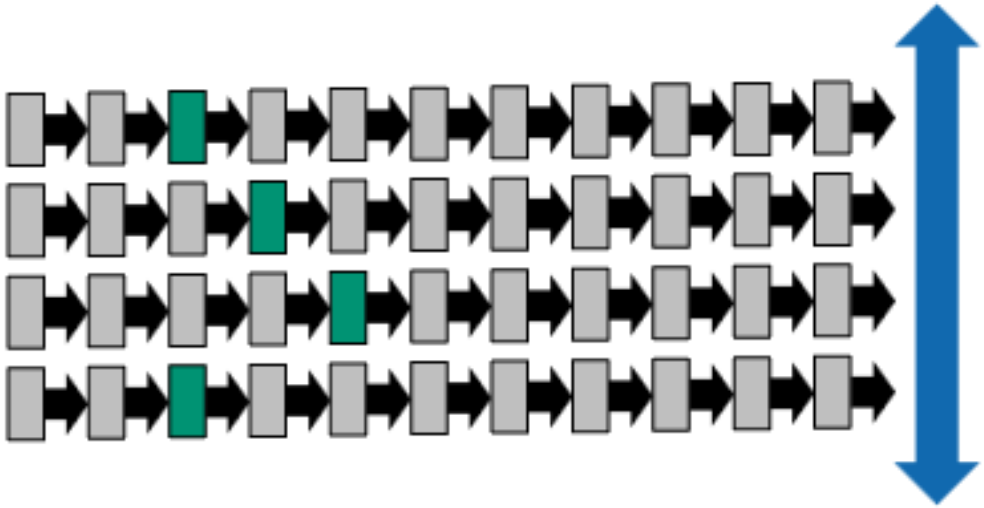
- Tri-word, 4-word, …



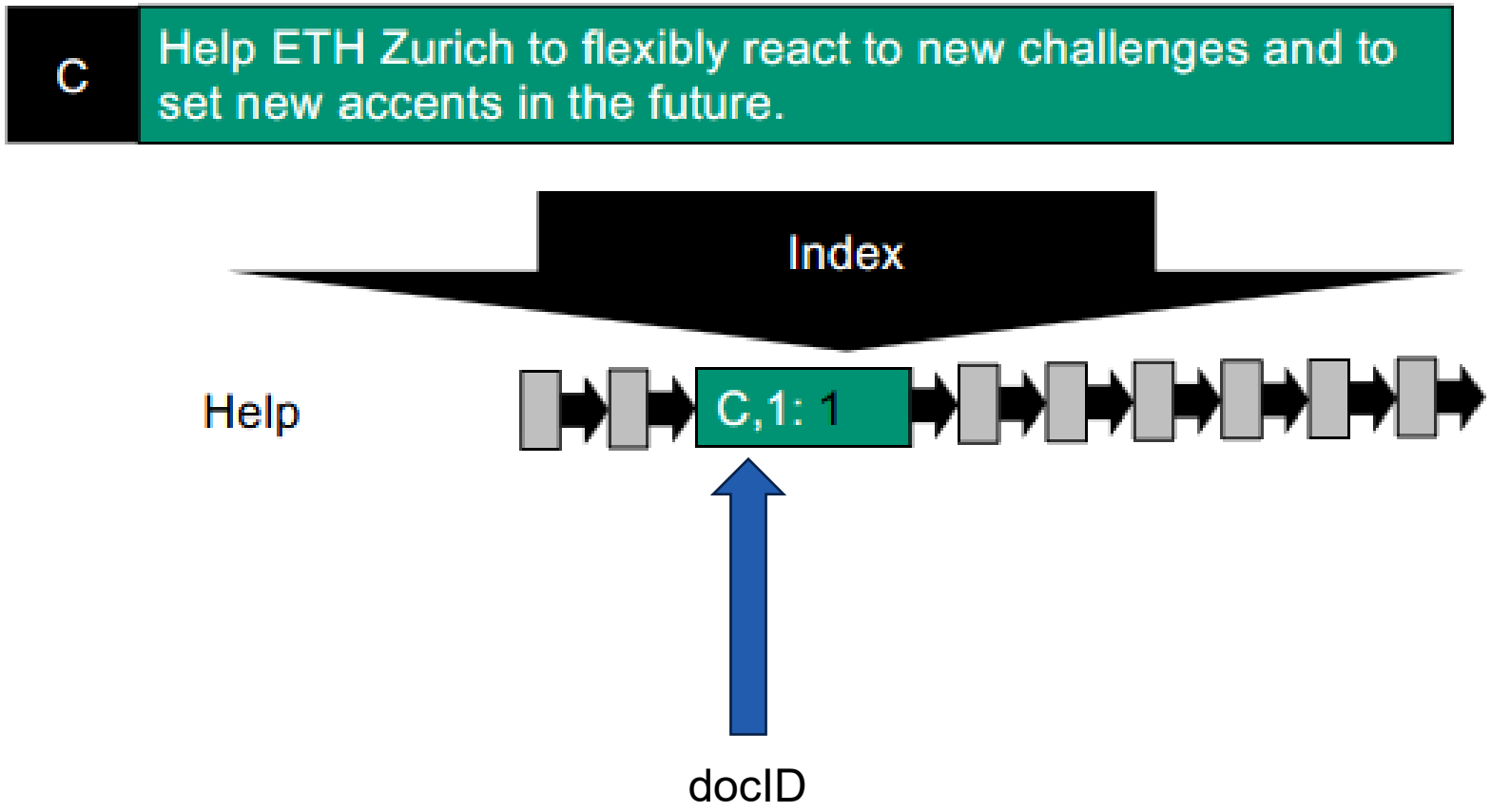


K-word indices

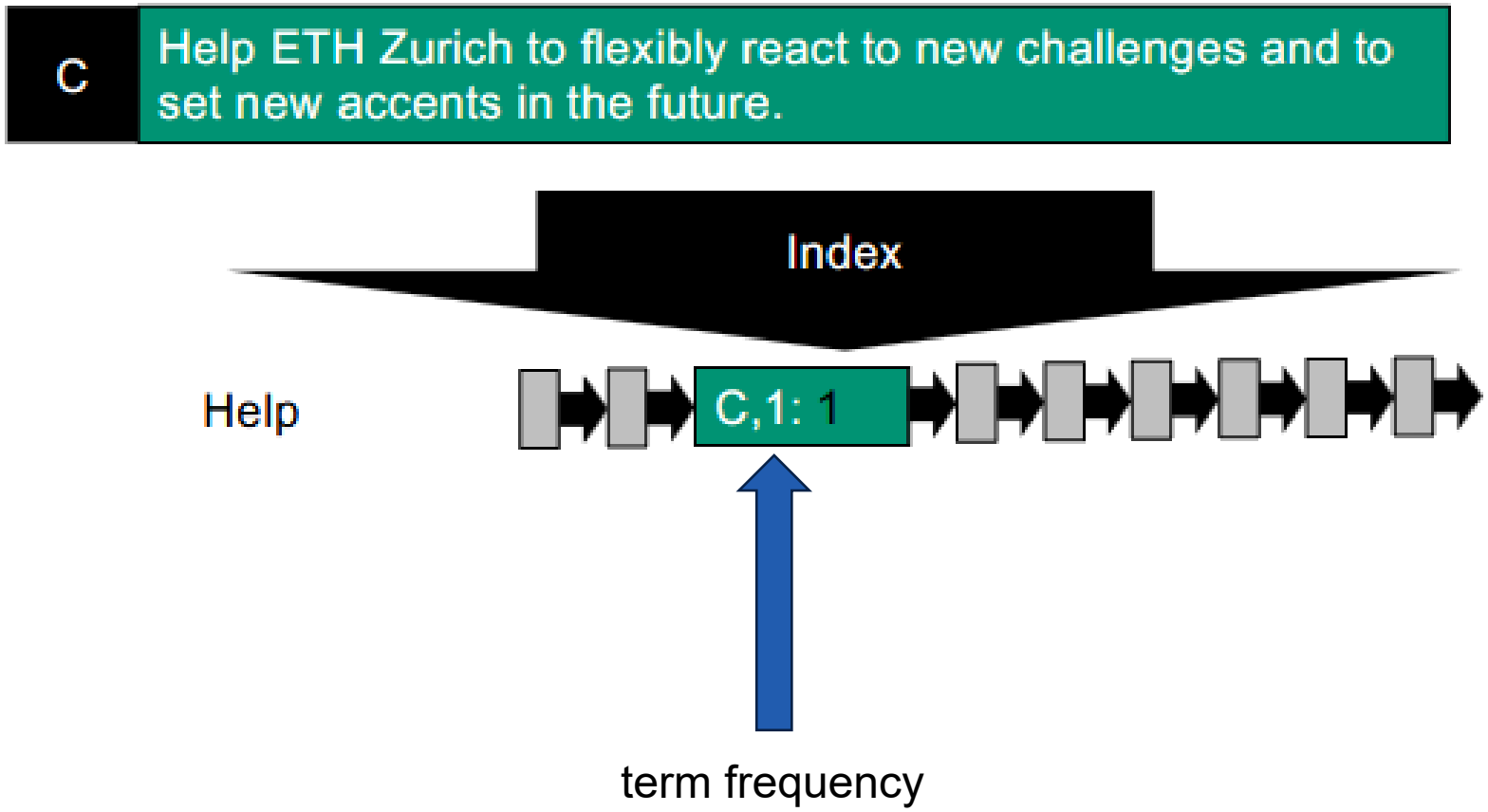
- Tri-word, 4-word, …
  - $(\#\mathrm{Terms})^{\mathrm{K}}$
  - Increases exponentially



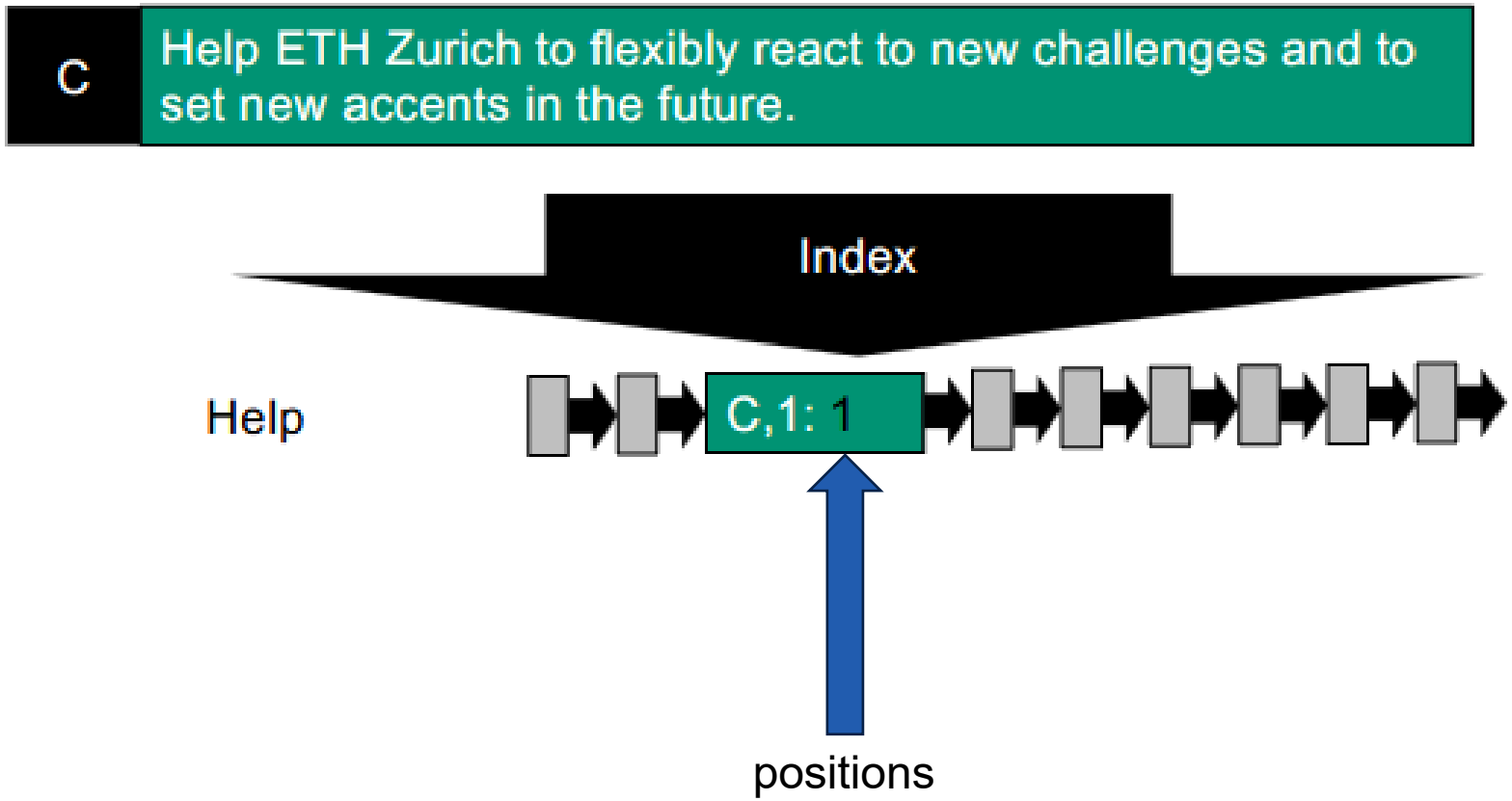
Positional index



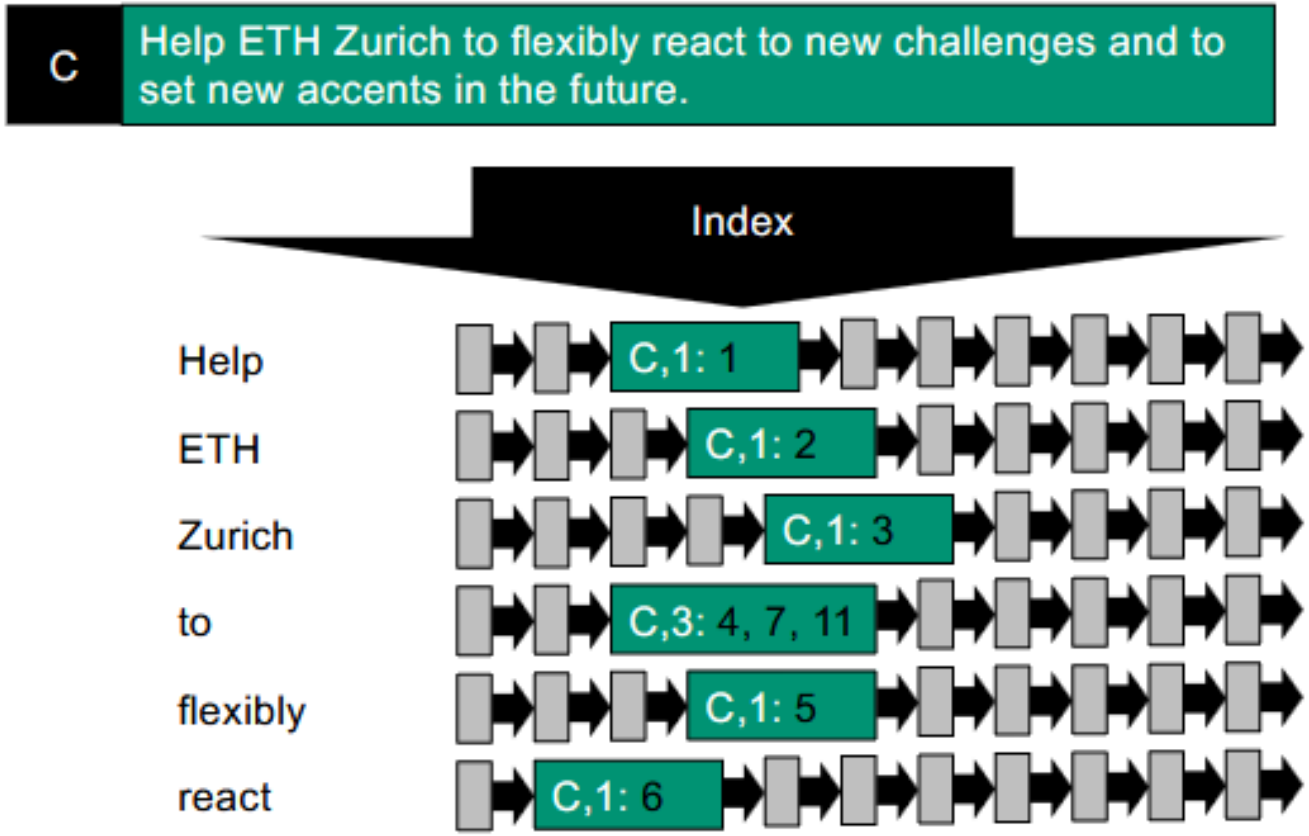
Positional index



Positional index



Positional index



Exercise 2: Advanced Indices

- Notebook
  - Part A: Implement k-word index
    - Modify *add_document()* and *execute_query()* to build k-word index and make phrase queries possible
  - Part B: Implement positional index
    - Modify *add_document()* and *positional_intersect_two()* to build index and process phrase queries
  - Part C: Theoretical questions about stemming
    - Porter Stemmer implemented in Notebook, answer theoretical questions on Moodle

Exam Questions

In a Boolean Retrieval system, stemming never lowers **precision**, assuming no NOT is used in the query.

False, e.g. «universe» and «university» both stem to «univers», so e.g. a query «university AND ranking» retrieves documents about the universe that happen to mention ranking. This is a false positive, lowering precision.

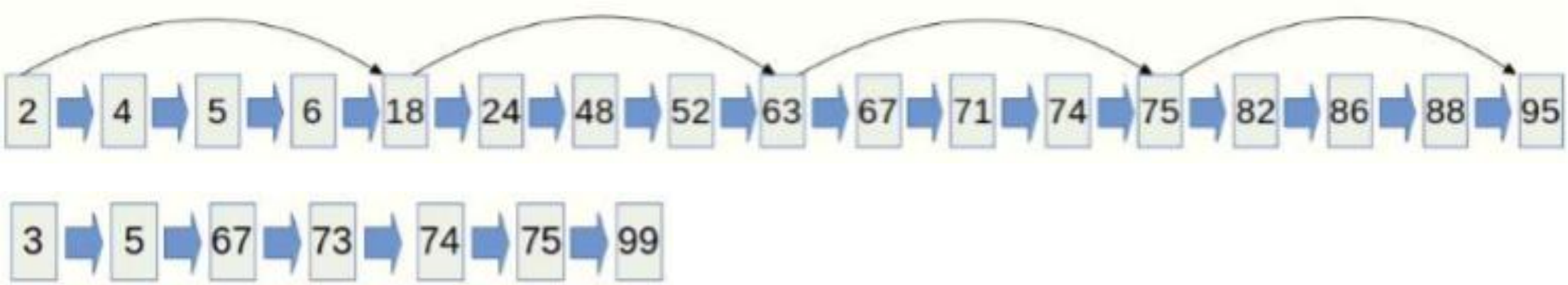
Exam Questions

In a Boolean Retrieval system, stemming never lowers **recall**, assuming no NOT is used in the query.

True. Stemming can only add matches, never remove them. NOT breaks this because it can cause a document to be excluded when it shouldn’t be.

Exam Questions

Consider the computation of the intersection of this postings list with skip pointers.

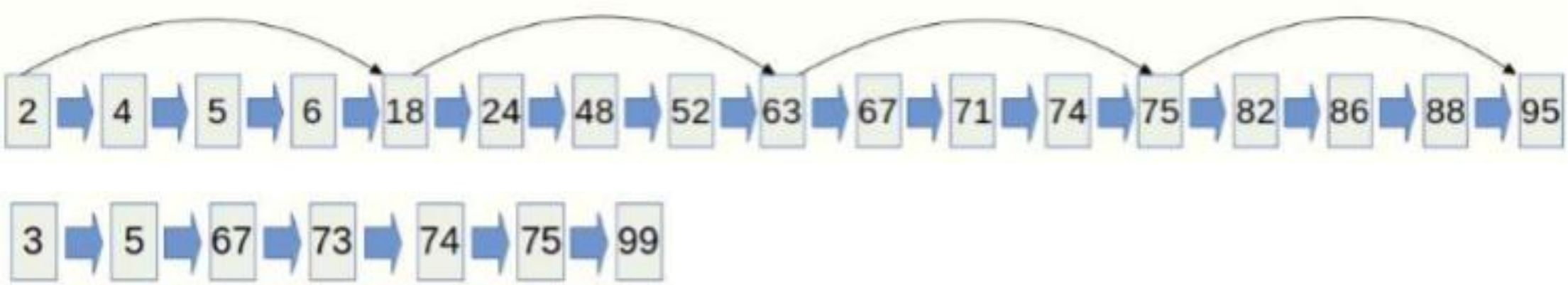


How many times is a skip pointer followed? (Note: when a comparison succeeds on a posting, the skip pointer is **not** considered for the next comparison).

1 (from 18 to 63).

Exam Questions

Consider the computation of the intersection of this postings list with skip pointers.



How many total comparisons of postings will be made? (Note: when a comparison succeeds on a posting, the skip pointer is **not** considered for the next comparison).

18

Exam Questions

Assume we have a bi-word index, no stemming, no lemmatization, no normalization. We consider the phrase query **«New York University»**. For each one of the following documents, mark whether it will be returned or not returned:

- ❑ New York University is also called NYU
- ❑ NYU is located in New York
- ❑ Welcome to the New students of York University
- ❑ York University is not located in New York

Exam Questions

Assume we have a bi-word index, no stemming, no lemmatization, no normalization. We consider the phrase query **«New York University»**. For each one of the following documents, mark whether it will be returned or not returned:

- ✓ New York University is also called NYU
- ❑ NYU is located in New York
- ❑ Welcome to the New students of York University
- ✓ York University is not located in New York

Exam Questions

Due to a natural catastrophe, some information was lost in the data center in which a library stored its documents and inverted indexes

In particular, they lost some parts of the document with ID 2. They now only know that it says:

... likes to ... and ... likes .....

Luckily, it was recorded in a positional index. But unfortunately, they also lost some parts of the index and only have the lists for the following terms:

she: **1**: [1, 5], **2**: [1], **3**: [3], **4**: [3], **5**: [1]

eat: **3**: [8], **2**: [4], **5**: [8]

pink: **4**: [8], **5**: [7]

recipe: **3**: [2]

cook: **1**: [4], **5**: [4]

Which document could potentially be document 2?

Note: We assume that the first word in the document is automatically capitalized and that full stop is added at the end of the document.

Exam Questions

Due to a natural catastrophe, some information was lost in the data center in which a library stored its documents and inverted indexes

In particular, they lost some parts of the document with ID 2. They now only know that it says:

... likes to ... and ... likes .....

Luckily, it was recorded in a positional index. But unfortunately, they also lost some parts of the index and only have the lists for the following terms:

she: **1**: [1, 5], **2**: [1], **3**: [3], **4**: [3], **5**: [1]

eat: **3**: [8], **2**: [4], **5**: [8]

pink: **4**: [8], **5**: [7]

recipe: **3**: [2]

cook: **1**: [4], **5**: [4]

Which document could potentially be document 2?

Note: We assume that the first word in the document is automatically capitalized and that full stop is added at the end of the document.

She likes to eat and he likes to eat

Exam Questions

Due to a natural catastrophe, some information was lost in the data center in which a library stored its documents and inverted indexes

In particular, they lost some parts of the document with ID 2. They now only know that it says:

... likes to ... and ... likes .....

Luckily, it was recorded in a positional index. But unfortunately, they also lost some parts of the index and only have the lists for the following terms:

she: **1**: [1, 5], **2**: [1], **3**: [3], **4**: [3], **5**: [1]

eat: **3**: [8], **2**: [4], **5**: [8]

pink: **4**: [8], **5**: [7]

recipe: **3**: [2]

cook: **1**: [4], **5**: [4]

Which document could potentially be document 2?

Note: We assume that the first word in the document is automatically capitalized and that full stop is added at the end of the document.

She likes to eat and he likes to eat

Not possible, since «eat» should only be present in index 4.

Exam Questions

Due to a natural catastrophe, some information was lost in the data center in which a library stored its documents and inverted indexes

In particular, they lost some parts of the document with ID 2. They now only know that it says:

... likes to ... and ... likes .....

Luckily, it was recorded in a positional index. But unfortunately, they also lost some parts of the index and only have the lists for the following terms:

she: **1**: [1, 5], **2**: [1], **3**: [3], **4**: [3], **5**: [1]

eat: **3**: [8], **2**: [4], **5**: [8]

pink: **4**: [8], **5**: [7]

recipe: **3**: [2]

cook: **1**: [4], **5**: [4]

Which document could potentially be document 2?

Note: We assume that the first word in the document is automatically capitalized and that full stop is added at the end of the document.

She likes to eat and he likes to wink

Exam Questions

Due to a natural catastrophe, some information was lost in the data center in which a library stored its documents and inverted indexes

In particular, they lost some parts of the document with ID 2. They now only know that it says:

... likes to ... and ... likes .....

Luckily, it was recorded in a positional index. But unfortunately, they also lost some parts of the index and only have the lists for the following terms:

she: **1**: [1, 5], **2**: [1], **3**: [3], **4**: [3], **5**: [1]

eat: **3**: [8], **2**: [4], **5**: [8]

pink: **4**: [8], **5**: [7]

recipe: **3**: [2]

cook: **1**: [4], **5**: [4]

Which document could potentially be document 2?

Note: We assume that the first word in the document is automatically capitalized and that full stop is added at the end of the document.

She likes to eat and he likes to wink

Possible, since there is no list for wink to disprove its position at index 9.

Exam Questions

Due to a natural catastrophe, some information was lost in the data center in which a library stored its documents and inverted indexes

In particular, they lost some parts of the document with ID 2. They now only know that it says:

... likes to ... and ... likes .....

Luckily, it was recorded in a positional index. But unfortunately, they also lost some parts of the index and only have the lists for the following terms:

she: **1**: [1, 5], **2**: [1], **3**: [3], **4**: [3], **5**: [1]

eat: **3**: [8], **2**: [4], **5**: [8]

pink: **4**: [8], **5**: [7]

recipe: **3**: [2]

cook: **1**: [4], **5**: [4]

Which document could potentially be document 2?

Note: We assume that the first word in the document is automatically capitalized and that full stop is added at the end of the document.

She likes to wink and he likes to eat

Exam Questions

Due to a natural catastrophe, some information was lost in the data center in which a library stored its documents and inverted indexes

In particular, they lost some parts of the document with ID 2. They now only know that it says:

... likes to ... and ... likes .....

Luckily, it was recorded in a positional index. But unfortunately, they also lost some parts of the index and only have the lists for the following terms:

she: **1**: [1, 5], **2**: [1], **3**: [3], **4**: [3], **5**: [1]

eat: **3**: [8], **2**: [4], **5**: [8]

pink: **4**: [8], **5**: [7]

recipe: **3**: [2]

cook: **1**: [4], **5**: [4]

Which document could potentially be document 2?

Note: We assume that the first word in the document is automatically capitalized and that full stop is added at the end of the document.

She likes to wink and he likes to eat

Not possible, since «eat» cannot be in index 9.

Exam Questions

Due to a natural catastrophe, some information was lost in the data center in which a library stored its documents and inverted indexes

In particular, they lost some parts of the document with ID 2. They now only know that it says:

... likes to ... and ... likes .....

Luckily, it was recorded in a positional index. But unfortunately, they also lost some parts of the index and only have the lists for the following terms:

she: **1**: [1, 5], **2**: [1], **3**: [3], **4**: [3], **5**: [1]

eat: **3**: [8], **2**: [4], **5**: [8]

pink: **4**: [8], **5**: [7]

recipe: **3**: [2]

cook: **1**: [4], **5**: [4]

Which document could potentially be document 2?

Note: We assume that the first word in the document is automatically capitalized and that full stop is added at the end of the document.

She likes to eat and he likes to drink

Exam Questions

Due to a natural catastrophe, some information was lost in the data center in which a library stored its documents and inverted indexes

In particular, they lost some parts of the document with ID 2. They now only know that it says:

... likes to ... and ... likes .....

Luckily, it was recorded in a positional index. But unfortunately, they also lost some parts of the index and only have the lists for the following terms:

she: **1**: [1, 5], **2**: [1], **3**: [3], **4**: [3], **5**: [1]

eat: **3**: [8], **2**: [4], **5**: [8]

pink: **4**: [8], **5**: [7]

recipe: **3**: [2]

cook: **1**: [4], **5**: [4]

Which document could potentially be document 2?

Note: We assume that the first word in the document is automatically capitalized and that full stop is added at the end of the document.

She likes to eat and he likes to drink

Possible, since there is no list to disprove that drink cannot be at index 9.

Exam Questions

For each of the following statements, mark whether it refers to the positional index or the biword index.

- This index has a larger number of postings lists than the other.
- The original documents (lists of words) can be fully reconstructured from the index. (For this statement, we consider than no post-processing beyond tokenization was performed to build the index.)
- Using this index for phrase search can lead to false positives.
- This index needs more space per term.

Exam Questions

For each of the following statements, mark whether it refers to the positional index or the biword index.

- This index has a larger number of postings lists than the other.
  - **Biword index**, since it can have a very large vocabulary and thus a very big number of postings lists.
- The original documents (lists of words) can be fully reconstructured from the index. (For this statement, we consider than no post-processing beyond tokenization was performed to build the index.)
- Using this index for phrase search can lead to false positives.
- This index needs more space per term.

Exam Questions

For each of the following statements, mark whether it refers to the positional index or the biword index.

- This index has a larger number of postings lists than the other.
  - **Biword index**, since it can have a very large vocabulary and thus a very big number of postings lists.
- The original documents (lists of words) can be fully reconstructured from the index. (For this statement, we consider than no post-processing beyond tokenization was performed to build the index.)
  - **Positional index**, since it saves the exact positions of the terms in the docs.
- Using this index for phrase search can lead to false positives.
- This index needs more space per term.

Exam Questions

For each of the following statements, mark whether it refers to the positional index or the biword index.

- This index has a larger number of postings lists than the other.
  - **Biword index**, since it can have a very large vocabulary and thus a very big number of postings lists.
- The original documents (lists of words) can be fully reconstructed from the index. (For this statement, we consider than no post-processing beyond tokenization was performed to build the index.)
  - **Positional index**, since it saves the exact positions of the terms in the docs.
- Using this index for phrase search can lead to false positives.
  - **Biword index**, as seen in the example in the lecture recap.
- This index needs more space per term.

Exam Questions

For each of the following statements, mark whether it refers to the positional index or the biword index.

- This index has a larger number of postings lists than the other.
  - **Biword index**, since it can have a very large vocabulary and thus a very big number of postings lists.
- The original documents (lists of words) can be fully reconstructured from the index. (For this statement, we consider than no post-processing beyond tokenization was performed to build the index.)
  - **Positional index**, since it saves the exact positions of the terms in the docs.
- Using this index for phrase search can lead to false positives.
  - **Biword index**, as seen in the example in the lecture recap.
- This index needs more space per term.
  - **Positional index,** since it not only saves the docIDs per term, but also the exact positions.

Kahoot

https://create.kahoot.it/details/f6631349-f63b-4c88-9cc2-c1bf9d140451